MANUAL TÉCNICO DEL PROGRAMADOR Y ARQUITECTURA DE CÓDIGO

Proyecto: Piedra, Papel o Tijera — Actividad 7
Institución: E.E.S.T. N.º 1 “Eduardo Ader” — Vicente López
Asignatura: Laboratorio de Programación (LPR) — 5.º año
Docentes: Mansilla Muñoz York Elías y Yamil Ganduglia
Autor: Lucas Benitez (Grupo 1)
Fecha: 14 de octubre de 2026
Versión: v1.0.0

CONTROL DE VERSIONES

Versión	Fecha	Autor	Descripción
v1.0.0	14/10/2026	Lucas Benitez	Versión inicial del manual técnico del programador para la Actividad 7.

1. ARQUITECTURA DEL SISTEMA Y ENTORNO DE DESARROLLO

1.1. Descripción general

El proyecto consiste en un programa de consola desarrollado en C + +. Su objetivo es repasar tres contenidos: recursividad, búsqueda secuencial en un arreglo e intercambio de valores mediante punteros. El código principal se encuentra en src/main.cpp.

1.2. Requisitos del entorno

- **Lenguaje: C + +.**
- **Compilador: g + +.**
- **IDE recomendado: Visual Studio Code.**
- **Terminal: Windows PowerShell.**
- **Bibliotecas utilizadas: iostream, fstream, string, cstdlib y ctime.**

1.3. Estructura del repositorio

Estructura de Directorios del Repositorio

piedrapapelotijera/

```
├── .gitattributes              <-- Normalización de finales de línea
├── .gitignore                  <-- Exclusiones (.exe, *.out, .vscode/)
├── LICENSE                     <-- Licencia MIT escolar
├── README.md                   <-- Portada del proyecto con integrantes
├── docs/                       <-- Entregables y documentación
│   ├── InformeEEST1_LPR2026_ACT07_G03_Informe_v1.0.0.pdf
│   ├──CHANGELOG.md             <-- Historial de versiones del software
│   └── manuales
│       ├── manual_programador_v1.0.0.pdf
│       ├── manual_programador_v1.0.0.md
│       ├── manual_usuario_v1.0.0.pdf
│       └── manual_usuario_v1.0.0.md
├── src/                        <-- Código fuente compilable
│   ├── main.cpp                <-- Programa principal del juego en C++
│   └── historial_partidas.txt  <-- Archivo de persistencia generado
└── capturas/                   <-- Evidencias de ejecución (.png)
    ├── jugada_consola.png      <-- Captura del menú y partida jugada
    └── archivo_disco.png       <-- Captura del archivo .txt abierto
```

2. DOCUMENTACIÓN DE MÓDULOS Y FUNCIONES (src/main.cpp)

2.1 Estructura Partida

- **Propósito:** agrupar los datos correspondientes a una partida.
- **Campos:** idPartida, jugador, eleccionJugador, eleccionIA y resultado.
- **Funcionamiento:** permite almacenar y transferir la información de cada partida mediante una única estructura.

2.2 Módulo 1: int main()

- **Propósito:** iniciar el programa y mostrar el menú principal.
- **Firma:** int main().
- **Lógica:** inicializa la generación aleatoria, presenta las opciones disponibles y ejecuta la función correspondiente según la elección del usuario. El programa continúa hasta seleccionar la opción de salida.

2.3 Módulo 2: void jugarPartida(int& contadorID)

- **Propósito:** gestionar una partida completa.
- **Firma:** void jugarPartida(int& contadorID).
- **Lógica:** solicita el nombre del jugador, recibe su elección, genera una jugada aleatoria para la computadora, determina el resultado y solicita guardar los datos de la partida.
- **Parámetro:** contadorID permite asignar identificadores consecutivos durante la ejecución.

2.4 Módulo 3: void guardarEnDisco(const Partida& p)

- **Propósito:** guardar los resultados de las partidas en un archivo de texto.
- **Firma:** void guardarEnDisco(const Partida& p).
- **Lógica:** abre src/historial_partidas.txt en modo de adición mediante ofstream e ios::app. Escribe los datos separados por comas y cierra el archivo.
- **Persistencia:** los registros anteriores se conservan al agregar nuevas partidas.

2.5 Módulo 4: void leerHistorialDisco()

- **Propósito:** recuperar y mostrar las partidas guardadas.
- **Firma:** void leerHistorialDisco().
- **Lógica:** abre el archivo con ifstream, lee los campos de cada registro y muestra el historial en la consola. Si el archivo no existe o no contiene registros, informa la situación al usuario.

3. INSTRUCCIONES DE COMPILACIÓN Y DEPURACIÓN

3.1. Compilación desde PowerShell

Ubicarse en la carpeta raíz del proyecto y ejecutar:

```
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe
```

3.2. Ejecución

```
.\src\piedrapapelotijera.exe
```

3.3. Diagnóstico de errores frecuentes

- **Error: g++ no se reconoce como comando.**
  Puede significar que el compilador no está instalado o que su carpeta bin no está incluida en PATH. Verificar la instalación de MinGW-w64/MSYS2 y abrir una terminal nueva.

- **Error al guardar o leer el historial.**

  La ruta relativa del archivo no coincide con la ubicación desde la que se ejecuta el programa. Ejecutar el programa desde la carpeta raíz del proyecto y comprobar los permisos de escritura.

- **El historial no aparece después de reiniciar el programa.**

  El archivo de historial no existe en la ruta esperada o fue eliminado. Verificar la ubicación de `src/historial_partidas.txt` y jugar una partida para generar nuevos registros.

4. REFERENCIAS

- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
- ISO. (2011). *ISO/IEC 14882:2011 — Programming Languages — C++*.